

Multi-Rail Isolated Power Supply Monitoring System

Capstone Design Documentation

Michael Capotosto

September 22, 2026

Rev. ET450-W2

Contents

1	Introduction	1
1.1	Project Overview	1
1.2	Project Motivation	2
1.3	System Architecture	2
1.4	Design Objectives	2
1.5	Project Scope	3
1.6	Report Organization	4
2	Preliminary Design Review	5
2.1	Purpose and Scope	5
2.2	Problem Definition	5
2.3	Proposed Solution	6
2.4	Chosen Conceptual Design and Functional Block Diagram	6
2.4.1	Functional Block Diagram	7
2.5	Preliminary Engineering Calculations	8
2.6	Functional Specifications	8
2.7	Design Methodology	8
2.8	Preliminary Design Schematics	9
2.8.1	Sensor Board Schematic	9
2.8.2	Arduino Shield Board Schematic	17
2.9	Software Architecture and Critical Software Test Results	19
2.10	Summary of Preliminary-Design Changes and Project Impact	19
3	Simulation Results	23

4	Role of Physics and Other Sciences	25
5	Installing EPICS Base and IOCs	27
5.1	Introduction	27
5.2	EPICS Installation Overview	27
5.3	Installing EPICS Base and Required Modules	27
5.4	Installing the PSU Monitor IOC	29
6	CS-Studio - Phoebus	31
6.1	Introduction	31
6.2	Installing Phoebus	31
7	Prototyping and Development	33
7.1	Introduction	33
7.2	Testing the LTC2945	33
7.2.1	Calculating Shunt Value for Test	34
7.2.2	Pictures of Evaluation Board, Display, and Phoebus Panels	35
A	Design Calculations and Circuit Analysis	41
A.1	Current-Sense Shunt Resistor Sizing	41
A.1.1	Current Measurement Resolution	42
A.1.2	Production Shunt Design Table	42
A.1.3	Evaluation-Board Shunt Calculation	42
A.2	PCB Trace and Copper-Pour Current Capacity	43
A.3	I ² C Pull-Up Resistor Sizing	43
A.4	LTC2945 Voltage Measurement Resolution	43
A.5	Negative-Rail Sign Convention	43
A.6	Alarm-Limit Scaling	44
A.7	Controller-Supply Power Budget	44
A.8	Component Capability Checks	45
A.9	Current-Sense Resistor Power Margin	45
A.10	LTC2945 Input and ADC Range	45
A.11	Microcontroller I/O Compatibility	45
A.12	Digital-Isolation Margin	45

A.13 3.3 V Regulator Loading	46
A.14 Tolerance and Measurement-Uncertainty Considerations	46
A.15 Reference Specifications Used for Capability Checks	46
B Detailed Software Flowchart	49
B.1 Arduino GIGA Firmware Flowchart	50
B.2 PSC and EPICS Communication Flow	52
B.3 Software Configuration Management	52
C Updated Component List	53
C.1 Bill of Materials	54
C.1.1 Sensor Board Bill of Materials	55
C.1.2 Arduino Shield Board Bill of Materials	56

Chapter 1

Introduction

1.1 Project Overview

The Multi-Rail Isolated Power Supply Monitoring System is a hardware and software system developed to provide centralized monitoring of multiple electrically isolated power-supply rails. The system measures both output voltage and load current, provides configurable alarm limits, displays operating status and measurement data via a local display or ethernet for integration with an EPICS-based control system.

The completed system monitors six independent power-supply rails:

- +5VA
- +5VB
- +5V
- -5V
- +15V
- -15V

Because the monitored supplies may be electrically isolated from one another, the monitoring system must preserve that isolation while still allowing all six channels to communicate with a common controller. Each supply rail therefore uses an independent voltage/current monitoring channel and an isolated digital communication path.

The primary controller is an Arduino GIGA R1 WiFi. The controller acquires measurement data from the individual monitor channels, converts the data to engineering units, applies the appropriate rail polarity, evaluates configured alarm limits, and presents the resulting information on a GIGA Display Shield. An Ethernet interface provides network communication between the embedded controller and an EPICS Input/Output Controller (IOC). The EPICS IOC then makes the measurements, alarms, and system status available to a CS-Studio Phoebus operator interface.

1.2 Project Motivation

Power-supply systems containing multiple isolated voltage rails present a monitoring challenge because each circuit associated with one supply can't share a direct electrical reference with another supply. Connecting conventional measurement or communication circuits across these references can break isolation between the rails.

A centralized monitor is desirable because it allows the user to observe all supply voltages and currents from a single interface, identify abnormal operating conditions, detect communication or sensor faults, and integrate the power-supply status into a larger control system.

The design developed for this project addresses these requirements by performing voltage and high-side current measurement locally on each monitored rail and transferring the resulting information across electrically isolated digital interfaces. The controller receives measurement data from all six rails without requiring their power references to be directly connected together.

1.3 System Architecture

The monitoring system is divided into three primary functional areas: the power-supply monitoring hardware, the microcontroller, and the supervisory control interface.

The power supply monitoring hardware contains the current-sense resistors, LTC2945 PMICs, and digital isolation circuitry. Each LTC2945 measures the supply voltage and the differential voltage developed across a high-side current-sense resistor. The resulting digital measurement information is sent over the I²C bus while maintaining the required electrical isolation.

The Arduino GIGA R1 WiFi provides the mcu processing functions. Its firmware initializes the monitoring channels, acquires voltage and current measurements, detects communication errors, converts raw measurements to engineering units, applies channel-specific polarity and scaling, evaluates alarm limits, and updates the local display. Configuration information, including network parameters and alarm limits, is retained in persistent storage so that required settings can survive a controller reset or power cycle.

Network communication is provided through an Ethernet interface using the PSC protocol. The EPICS IOC communicates with the embedded controller using PSCDriver and converts the transmitted values into EPICS process variables. CS-Studio Phoebus provides the remote operator interface for viewing measurements, alarm conditions, connection status, and system state.

1.4 Design Objectives

The major design objectives for the Multi-Rail Isolated Power Supply Monitoring System are to:

1. Measure the voltage of six independent power-supply rails.
2. Measure the load current of each rail using high-side current sensing.
3. Preserve electrical isolation between independently referenced power-supply rails and the common controller.
4. Convert acquired data into meaningful engineering units and correctly represent both positive and negative voltage rails.
5. Detect I²C communication failures and prevent invalid sensor data from being presented as valid measurements.
6. Compare voltage and current measurements against configurable upper and lower alarm limits.
7. Provide a local graphical display of voltage, current, alarm status, communication status, and network state.
8. Retain configuration and alarm settings through controller resets and power cycles.
9. Provide Ethernet communication for remote monitoring and configuration.
10. Integrate the monitoring system with an EPICS IOC and CS-Studio Phoebus operator interface.
11. Maintain the hardware, firmware, and control-system software under revision control so that changes made during design, implementation, and testing remain traceable.

1.5 Project Scope

The scope of the project includes the design and implementation of the sensor interface hardware, Arduino GIGA controller hardware, embedded firmware, Ethernet communication interface, EPICS IOC, and operator-interface software. The hardware portion includes the six voltage/current monitoring channels, current-sense resistors, digital isolation circuitry, controller interface, Ethernet circuitry, and supporting power-supply circuitry.

The software portion includes the embedded acquisition and alarm-processing firmware, persistent configuration storage, local display interface, PSC network communication, EPICS IOC configuration and database records, and the CS-Studio Phoebus operator interface.

Design files and software associated with the project are maintained in the project Git repository:

https://github.com/capotosto/PS_Monitor

The repository provides revision control for the hardware design, firmware, EPICS IOC files, documentation, and other project artifacts generated during development.

1.6 Report Organization

This report documents the development of the monitoring system from preliminary design through implementation and testing. The preliminary-design chapter defines the major functional blocks, presents the system architecture, documents supporting engineering calculations, includes the hardware schematics and bills of materials, and describes the primary firmware and control-system software flows.

Subsequent chapters document prototyping and development activities, the EPICS IOC implementation, and the CS-Studio Phoebus operator interface. Supporting project-management and design documents developed during the earlier phases of the capstone project are retained as part of the overall project documentation and provide traceability between the original requirements, design decisions, implementation, and final system.

Chapter 2

Preliminary Design Review

2.1 Purpose and Scope

This chapter documents the preliminary hardware and software design of the Multi-Rail Isolated Power Supply Monitoring System. The design monitors six power-supply rails, measures rail voltage and load current, preserves electrical isolation between floating supplies and the controller, evaluates measurements against user-configurable alarm limits, presents local status on the controller display, and publishes measurements and status to an EPICS control-system interface.

This preliminary design is intended to establish the major analog and digital functional blocks, document supporting engineering calculations, define the hardware schematic base-line, and provide software flowcharts detailed enough to support implementation. Simulation and additional verification results can be added in the subsequent design phase.

2.2 Problem Definition

The project addresses the need to monitor several low-voltage power-supply rails that do not necessarily share a common reference. A conventional monitoring circuit that directly connects the grounds together would defeat the electrical isolation of the supplies and could create unwanted return paths. However, the user needs a single location from which to observe rail voltage, rail current, alarm state, communication status, and the overall condition of the power system.

The required monitoring system must therefore make accurate measurements on six separate rails while preserving isolation between the monitored domains. It must support both positive and negative rails, detect sensor or communication faults, provide configurable voltage and current limits, retain configuration information after reset, present useful local status, and make the same data available to the facility control system over Ethernet. The monitored rails in the present design are +5 V A, +5 V B, +/-5 V, and +/-15 V.

2.3 Proposed Solution

The proposed solution assigns one LTC2945 voltage/current monitor to each power rail. Load current is converted to a small differential voltage by a precision high-side shunt resistor and the LTC2945 digitizes both the shunt voltage and the rail voltage. Each sensor remains referenced to its own monitored supply domain. The I²C communication path is then isolated so that digital measurement data can be transferred to a common Arduino GIGA R1 WiFi controller without directly bonding the isolated supply returns together.

The Arduino GIGA performs channel acquisition, engineering-unit conversion, negative-rail sign handling, alarm evaluation, configuration storage, display updates, and network communication. A local GIGA Display Shield provides immediate status at the hardware. Ethernet transports measurement and status data using the PSC protocol to an EPICS IOC, where PSCDriver and EPICS records make the values available to a CS-Studio Phoebus operator interface. This architecture separates the rail-side analog measurement functions from the controller and supervisory-control functions while maintaining a single user interface for the complete system.

2.4 Chosen Conceptual Design and Functional Block Diagram

The major system functions are summarized below.

1. **Power-Supply Interface:** Connect to the six monitored rails and route each rail through the associated voltage/current monitoring channel.
2. **Current Measurement:** Convert load current to a differential voltage across a precision shunt resistor and digitize it with the LTC2945.
3. **Voltage Measurement:** Measure the monitored rail voltage using the LTC2945 supply-voltage ADC channel.
4. **Electrical Isolation:** Maintain galvanic isolation between each floating monitored supply and the common controller-side I²C bus.
5. **Digital Acquisition:** Poll each monitor, identify communication faults, apply channel-specific sign/scaling information, and calculate engineering-unit values.
6. **Alarm Processing:** Compare voltage and current values against configured upper and lower limits and generate channel alarm states.
7. **Local User Interface:** Present all rail measurements, alarms, network state, and communication status on the Arduino GIGA Display.
8. **Network/EPICS Interface:** Publish measurements, alarm limits, alarm state, and connection status over Ethernet for the EPICS IOC and Phoebus interface.

2.4.1 Functional Block Diagram

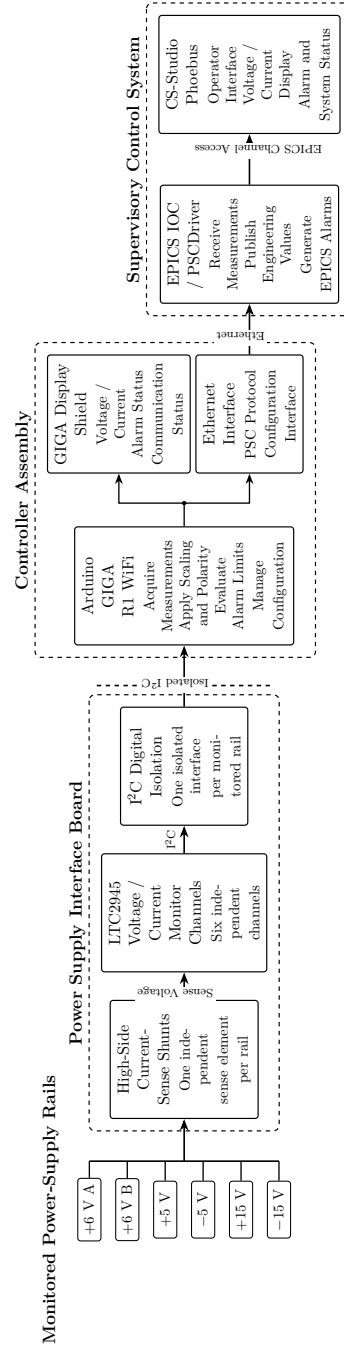


Figure 2.1: Preliminary functional block diagram of the multi-rail isolated power supply monitoring system. Six independent power-supply rails are measured using high-side current sensing and LTC2945 voltage/current monitoring channels. Digital isolation preserves electrical independence between the monitored supplies and the common controller. The Arduino GIGA processes the measurements, provides a local operator display, and transmits measurement and status information over Ethernet to the EPICS IOC and CS-Studio Phoebus operator interface.

2.5 Preliminary Engineering Calculations

2.6 Functional Specifications

The functional specifications used for the preliminary design are summarized in Table 2.1. These values define the intended operating envelope of the prototype and establish the quantities that must be checked during circuit analysis and later verification testing.

Table 2.1: Preliminary functional specifications.

Function	Specification
Monitored rails	+5 V A, +5 V B, +5 V, -5 V, +15 V, and -15 V
Number of channels	Six independent voltage/current measurement channels
+5 V A/B current range	0 to 2 A design range per rail
+5 V, -5 V, +15 V, -15 V current range	0 to 1 A design range per rail
Voltage measurement resolution	25 mV/LSB nominal using the LTC2945 rail-voltage ADC
Current-sense resolution	25 μ V/LSB across the selected shunt resistor; approximately 0.641 mA/LSB with 39 m Ω and 0.500 mA/LSB with 50 m Ω
Isolation	Independent digital isolation for every floating monitor channel
Alarm processing	User-configurable upper/lower voltage and current limits with local and EPICS status indication
Fault handling	Detect failed I ² C transactions and prevent invalid data from being treated as a valid measurement
Local interface	GIGA Display Shield showing voltage, current, alarm state, communication state, and configured limits
Network interface	Ethernet PSC server for EPICS IOC communication plus browser-based configuration/monitoring services
Configuration retention	Network parameters and alarm limits stored in nonvolatile memory
Controller logic level	3.3 V; external interfaces must not exceed the GIGA I/O limits

2.7 Design Methodology

The design was developed by dividing the system into measurement, isolation, controller, user-interface, and control-system functions and then verifying the interfaces between those blocks. The analog measurement design began with the maximum required rail current. The shunt value was selected so that the maximum nominal current produces a differential voltage comfortably below the LTC2945 102.4 mV current-sense full-scale value. Power dissipation in each shunt was then calculated using $P = I^2R$, and the selected resistor rating was checked against the calculated worst-case dissipation. The same approach was used for PCB

copper: maximum continuous current, allowable temperature rise, and voltage drop were considered before final trace and copper-pour dimensions were chosen. Detailed calculations are included in Appendix A.

Electrical isolation was treated as a system-interface requirement rather than only a component-selection problem. Each LTC2945 remains electrically local to its rail, and the digital signals cross the isolation boundary through isolated logic. Controller-side I²C buffering and level compatibility were checked against 3.3 V logic requirements. The controller does not drive loads directly; its GPIO and I²C pins only interface to logic-level circuitry. This substantially reduces the current demanded from an individual microcontroller pin and allows the high-current rail paths to remain on the dedicated sensor hardware.

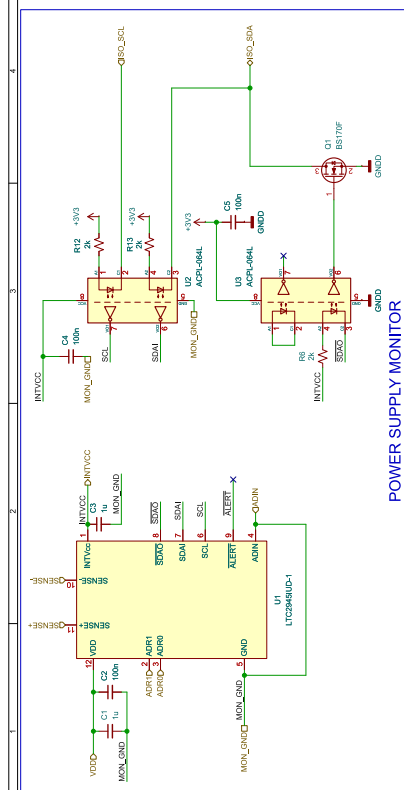
The embedded-software design follows the same functional partitioning. Sensor acquisition is separated from display formatting, network services, persistent storage, and PSC message handling. Alarm processing is performed on engineering-unit values after channel polarity and scaling are applied. The EPICS IOC is intentionally kept outside the embedded controller so that the Arduino is responsible for hardware acquisition while the IOC is responsible for exposing process variables to the remote user control system.

2.8 Preliminary Design Schematics

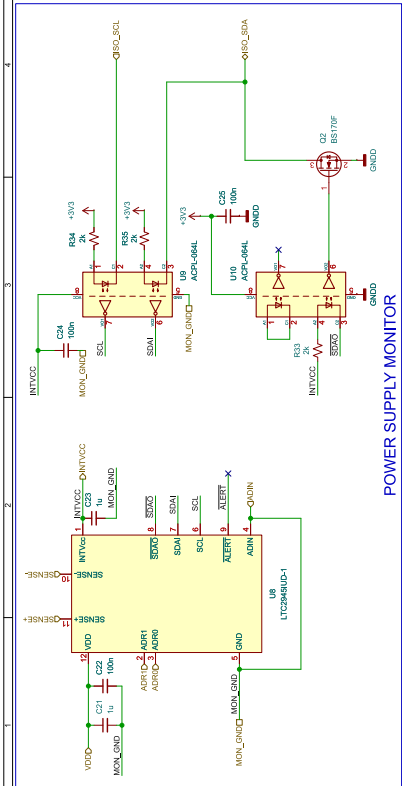
The preliminary hardware schematics are maintained in KiCad and are included below directly from the plotted PDF files in the project hardware directory.

2.8.1 Sensor Board Schematic



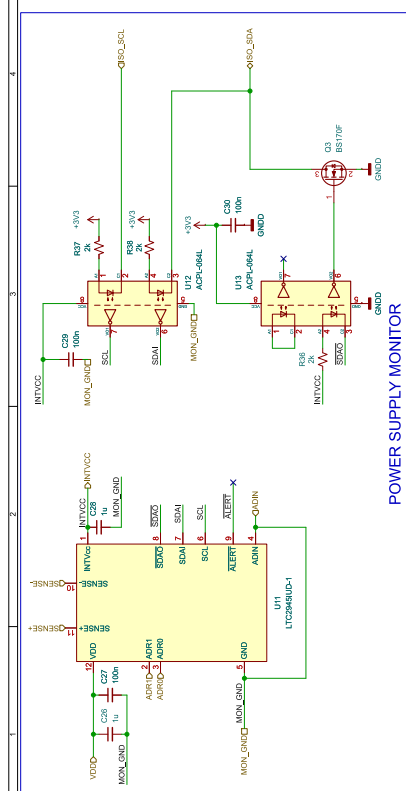


POWER SUPPLY MONITOR



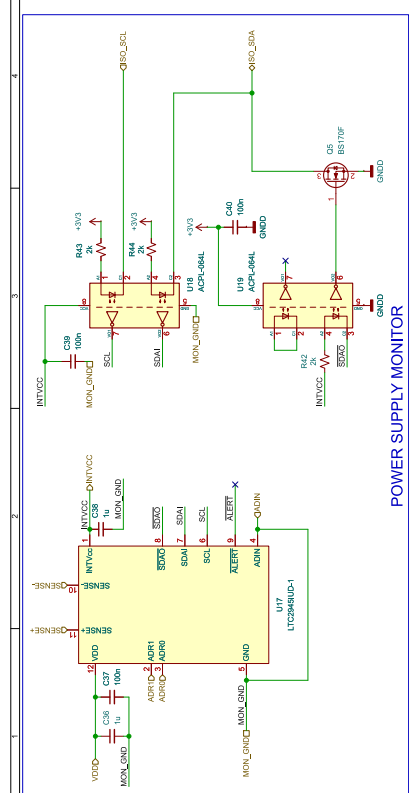
POWER SUPPLY MONITOR

CGS-PSM-03



POWER SUPPLY MONITOR







2.8.2 Arduino Shield Board Schematic



2.9 Software Architecture and Critical Software Test Results

The embedded software is organized into functional modules rather than one loop. The current implementation contains shared application state, sensor-management and LTC2945 register-access code, display handling, Ethernet services, an web control page, PSC protocol handling, persistent storage, and the main application controller. This separation permits the measurement code to be tested independently of the EPICS IOC and permits the network/control-system interface to consume already processed engineering-unit values.

At startup, the firmware initializes hardware interfaces, loads persistent settings, configures the monitoring channels, and then enters repetitive sensor acquisition and service processing. Each channel read is checked for successful I²C communication before conversion. Valid raw ADC values are scaled using the configured shunt resistance and channel polarity; invalid transactions produce a communication error state. Voltage and current values are then compared with their configured lower and upper limits before the local display and PSC message data are updated. The detailed execution flow is shown in Appendix B.

The most critical functions were tested to the extent possible on the available single-channel evaluation board setup. Sensor acquisition returned stable voltage and current data, open/unconnected channels produced I²C fault states instead of plausible but invalid values, negative-rail sign handling was visible on the -5 V test channel, and changing a voltage limit changed the local alarm state as expected. Ethernet/PSC communication was verified by the EPICS IOC reporting a connected driver and by Phoebus displaying the same test-channel voltage/current data. Remaining multi-channel verification, long-duration operation, restart/persistence checks, and final PCB testing will be repeated on the assembled production hardware. This is shown in more detail in Prototyping and Development

2.10 Summary of Preliminary-Design Changes and Project Impact

The preliminary-design work identified several items that were clarified or changed from the earliest project documentation. Table 2.2 summarizes the principal changes and their effect on the design baseline.

Table 2.2: Week 2 design changes and resulting impact.

Item	Preliminary-design result	Impact
Rail naming	The two high-current digital rails are +5 V A and +5 V B.	Some documentation and channel labels were corrected where it showed +6V; electrical architecture is unchanged.
Production shunts	39 m Ω selected for 2 A +5 V rails and 50 m Ω selected for 1 A rails.	Improves headroom and keeps shunt dissipation well below the selected resistor rating.
Prototype shunt	The DC1697A shunt was changed to approximately 17.8 Ω for low-current Arduino-driven loads.	Allows realistic ADC utilization during testing without exceeding Arduino regulator current limits.
I ² C pull-ups	External 4.7 k Ω pull-up footprints remain available but are DNP where the selected buffer/current-source arrangement makes them unnecessary.	Reduces unnecessary loading and preserves the intended I ² C interface behavior.
Fault handling	Missing or failed sensors are represented as communication faults rather than numerical measurements.	Reduces risk of a supervisory system accepting stale or fabricated values.
Configuration and network software	Alarm/network settings, HTTP services, PSC communication, and the Phoebe interface are now part of the implemented software baseline.	Adds firmware integration work but reduces final-system risk because these interfaces can be exercised before all six production channels are assembled.
Cost/schedule baseline	No formal budget or schedule re-baseline is required solely because of the Week 2 design changes.	Component values and firmware structure changed within the existing architecture; procurement cost will be rechecked after first-article PCB testing and any resulting BOM revisions.

The largest remaining technical risk is integration of all six isolated channels on the final PCB rather than operation of any single functional block. The single-channel prototype reduced uncertainty in the LTC2945 measurement, firmware scaling, alarm processing, Ethernet/PSC firmware, and operator interface. Final verification will focus on channel-to-channel isolation,

concurrent I²C operation, and confirmation that all configured settings survive normal resets and power cycles.

Chapter 3

Simulation Results

No simulations were performed for this project; instead, calculations were performed, and preliminary tests carried out with an evaluation board.

Chapter 4

Role of Physics and Other Sciences

To measure the currents of the various supplies, the load current is passed through a shunt resistor for each supply. Ohm's law, $V = IR$, converts the sampled current into a voltage that can be measured by the LTC2945 PMIC. For example, the digital rail's 2 A current flowing through a 39 m Ω shunt produces a 78 mV differential voltage. The resistor also converts part of the electrical energy into heat according to Joule heating, $P = I^2R$. That same 2 A operating point dissipates 0.156 W in the shunt. These two equations create the main current-sensor design tradeoff: a larger resistance produces more measurable voltage and better current resolution, but also produces greater voltage drop and heat. The selected shunt must therefore provide adequate ADC signal while remaining comfortably below its power rating and while causing acceptably small voltage/IR drop to the monitored rail.

Voltage measurement relies on the physics of electric potential difference and on analog-to-digital (ADC) conversion. Each LTC2945 is referenced to its own local power-supply domain via optocouplers, so it can measure the potential of that rail without requiring all six rails to share a common ground. The converter represents an analog input with a finite number of digital states. With the LTC2945's 25 mV-per-count resolution, quantization sets the bar for the lower measurable limit. The current channel similarly resolves 25 μ V per count across the shunt. Mathematics is required not only to convert the ADC code into volts and amps but also to determine whether the selected shunt and expected signal range make use of the full ADC span. The calculations in Appendix A explicitly convert between current, shunt voltage, ADC counts, and measurement resolution.

Galvanic isolation is another important physical concept in the project. Two circuits can exchange information without sharing a conductive DC path by using an isolator. In the ACPL-064L, an electrical input drives an LED, which is then detected by a photodiode on the other side, forming an isolated barrier between the two sides of the circuit. The signal crosses the isolation barrier while the two circuit grounds remain electrically independent. This is important because directly connecting the returns of floating supplies can create ground loops or force current through a path that the original power system did not intend. The mathematical treatment of the isolation interface includes logic voltage limits, LED/interface current, and the allowable voltage across the isolation barrier. Although the monitored rails are only in the tens of volts, the isolation components have ratings in excess of 100V,

providing ample headroom.

The digital communication buses also depend on basic circuit behavior. An I²C bus uses open-drain I/O, meaning that devices pull a line low while a pull-up element or active current source returns it high. Bus capacitance and pull-up strength determine the line's electrical rise time. Pull ups that are too weak, or too much bus capacitance can prevent a logic-high level from being reached quickly enough at the selected clock rate. The TCA9803 buffer used in the design includes an active current source on one side, which eliminates the need to be concerned with setting RC time constants so long as bus capacitance remains below the max for the driver.

Power conversion on the controller hardware also uses electromagnetic energy storage. The TPS62912 is a switching buck regulator: it alternately connects and disconnects the input through switching devices while an inductor and capacitors store and release energy to produce a lower average DC output. The basic relationships among voltage, current, inductance, capacitance, switching frequency, ripple, and load determine output quality. For this project the regulator is required to produce a 3.3 V logic rail from a higher input while supplying the display, Ethernet, and interface electronics. The preliminary load estimate is compared with the regulator's output-current rating so the design has sufficient current and thermal margin rather than selecting the regulator only from its nominal output voltage.

Finally, the project relies on mathematics to turn these physical principles into engineering decisions. Kirchhoff's and Ohm's laws are used to solve the prototype test network; $P = VI$, $P = I^2R$, and related equations are used for component heating and power budgeting; ADC scale factors are used to convert between raw counts and engineering units; inequalities define acceptable alarm regions; and tolerance analysis estimates how resistor error and converter error propagate into the reported current. The firmware then implements these same relationships numerically. In this sense the software is not separate from the physics of the hardware: it is the mathematical model that converts measured electrical quantities into the voltage, current, alarm, and status information presented to the user and to EPICS.

Chapter 5

Installing EPICS Base and IOCs

5.1 Introduction

A simple IOC was written to interface the Arduino Giga which monitors the six supplies. The following section details how to install EPICS Base and required modules, and compile the IOC. It is written assuming a Linux environment; this particular build was done on Ubuntu 22.04LTS.

5.2 EPICS Installation Overview

EPICS installations performed by M. Capotosto loosely follow the below directory structure:

- `EPICS_BASE=/epics/base` or `/epics/base_[version]` or similar
- `SUPPORT=/epics/support` – or sometimes `MODULES` instead of `SUPPORT` is used.

Modules required for the PSU monitor are:

- `pscdrv`: Portable Streaming Controller Driver
- `epics/iocs` - each IOC is cloned into this repository
- `epics/systemd-softioc` - This is a utility for managing IOCs as a system service

5.3 Installing EPICS Base and Required Modules

1. Install Dependencies for Build Install git, build-essential, and perl:

```
sudo apt -y install git build-essential perl libtirpc-dev re2c libevent-dev
autoconf automake libtool pkg-config
```

2. Create the EPICS user group, and the empty EPICS directory:

```
sudo groupadd epics
sudo usermod -aG epics $USER
newgrp epics
sudo mkdir -p /epics
sudo chown $USER:epics /epics
sudo chmod 2775 /epics
```

3. EPICS Base is cloned into /epics/base via:

```
git clone -recursive -b 7.0
https://github.com/epics-base/epics-base.git /epics/base
cd /epics/base
```

4. Build EPICS:

```
make clean
make
```

5. Create the remaining directory structure:

```
mkdir -p /epics/iocs /epics/modules
```

6. Clone PSC Driver:

- (a) `cd /epics/modules`
- (b) `git clone https://github.com/mdavidsaver/pscdrv.git pscdrv`
- (c) `cd /epics/modules/pscdrv`
- (d) Set up required files in `configure/RELEASE`, `configure/RELEASE.LOCAL`
- (e) Build with `make clean`, and `make`

7. Clone Manage IOCs Utility:

- (a) `cd /epics`
- (b) `git clone https://github.com/NSLS2/systemd-softioc.git`
- (c) Navigate to the repo directory `cd systemd-softioc`
- (d) Make the install script executable and run:


```
sudo chmod +x install.sh
sudo ./install.sh
sudo usermod -aG epics softioc
newgrp
sudo chown softioc:epics /epics/iocs
sudo chmod 2775 /epics/iocs
```

5.4 Installing the PSU Monitor IOC

1. Change to IOC directory `cd /epics/iocs`
2. Clone the repo:
`git clone https://github.com/capotosto/PS_Monitor psuMonitorIOC`
3. `cd psuMonitorIOC`
4. Create the `RELEASE.local` file, `touch configure/RELEASE.local`
5. `echo -e "EPICS_BASE = /epics/base\nPSCDRV = /epics/modules/pscdrv\n"`

6. Build the IOC: `make clean, make`
7. Update the `st.cmd` file as needed: `cd iocBoot/iocpsuMonitor`
8. Create the `st.cmd` file used by `systemd-softioc`:

```
echo -e '#!/bin/bash\n\n
cd "/epics/iocs/psuMonitorIOC/iocBoot/iocpsuMonitor"\n./st.cmd' > st.cmd
```

9. Make the new `st.cmd` executable: `chmod +x st.cmd`
10. Find the next available port: `manage-iocs nextport`
11. Configure this in the `config` file: `nano config`
12. Set the `NAME=`, `PORT=` your next port, `USER=softioc`, and `HOST=$HOSTNAME`
13. Update the directory path in the root-level `st.cmd` to the `iocBoot/iocpsuMonitor/st.cmd`
14. Enroll in `manage-iocs` as a service: `sudo manage-iocs install psuMonitorIOC`
15. Enable auto-start on boot: `sudo manage-iocs enable psuMonitorIOC`
16. Check the status, it should be running! `manage-iocs status`
17. And if you want other information on the ioc: `manage-iocs report`

Chapter 6

CS-Studio - Phoebus

6.1 Introduction

CS-Studio Phoebus is installed using precompiled binaries. This application requires a Java runtime environment be installed.

The 'BOB' files - the user interface panels, are located in the IOC's directory, `/epics/iocs/PS_Monitor/opi`

Behavior of some features of Phoebus must be modified by the shell script used to launch Phoebus, an example is included with the below installation instructions to modify the behavior of the 'Home' button, used to bring up the custom 'psuMonitor.bob' panel using the Home button, instead of having to manually navigate to the file when needed.

6.2 Installing Phoebus

1. If not installed, install Java, e.g.:
`sudo apt update && sudo apt install openjdk-17-jdk`
2. Create the directory: `sudo mkdir /opt/phoebus`
`sudo chown $USER:epics /opt/phoebus`
`chmod 2775 /opt/phoebus`
3. Download Phoebus and extract to the `/opt/phoebus` directory
<https://github.com/ControlSystemStudio/phoebus/releases/download/v4.7.4/phoebus-4.7.4-linux.tar.gz>

```
cd /opt/phoebus
```

```
wget https://github.com/ControlSystemStudio/phoebus/releases/download/  
v4.7.4/phoebus-4.7.4-linux.tar.gz
```

```
tar -xvzf phoebus-4.7.4-linux.tar.gz -C . -strip-components=1
```

4. Make the shell script executable: `chmod +x phoebus.sh`
5. Alias Phoebus to run via command `run-phoebus`:

```
echo 'alias run-phoebus="/opt/phoebus/phoebus.sh"' » ~/.bashrc  
source ~/.bashrc
```
6. Now test! Try `run-phoebus` from a terminal.

Chapter 7

Prototyping and Development

7.1 Introduction

7.2 Testing the LTC2945

The LTC2945 testing was carried out using a Linear Tech eval board, DEMO CIRCUIT 1697A. Connections and modifications were made as follows:

- VIN: tied to Arduino +5V
- GND: tied to Arduino GND
- ADIN: Tied to GND
- SCL: Tied to Arduino Pin D8 I2C2 SCL
- SDAI/SDAO (JP2 set to TIE): Arduino Pin D9 I2C2 SDA
- VPU: Tied to Arduino +3v3
- VOUT: Float or tied to dummy load ($1\text{k}\Omega$ to $10\text{k}\Omega$)
- $20\text{m}\Omega$ shunt replaced with 17Ω shunt for testing with $\sim 5\text{k}\Omega$ load
- Installed $4.7\text{k}\Omega$ pull up on I2C lines via R4, R5 footprints on EVB

7.2.1 Calculating Shunt Value for Test

Using the 5V from the arduino as a source, and a load ranging 1k to 10k ohms, we calculate:

$$R_{L,min} = 1k\Omega$$

$$V_{shunt} = V_S \frac{R_S}{R_L + R_S}$$

$$R_S = \frac{V_{shunt} R_L}{V_s - V_{shunt}}$$

$$V_{shunt} = 0.1024V$$

$$R_{S,fullscale} = \frac{(0.1024V)(1k\Omega)}{5V - 0.1024V} = 20.91\Omega$$

Adding some headroom, say, 15%, and going to an E96 resistor value...

$$R_S = 0.85(20.91) = 17.8\Omega$$

Verifying for the high end of the range:

$$I_{min} = \frac{5V}{10k\Omega + 17.8\Omega} = 499.1\mu A$$

$$V_{shunt,min} = (499.1\mu A)(17.8\Omega) = 8.88mV$$

The shunt will range from 8.88mV to 87.44mV, within the ADC's usable range of 0V to 102.4mV.

7.2.2 Pictures of Evaluation Board, Display, and Phoebus Panels

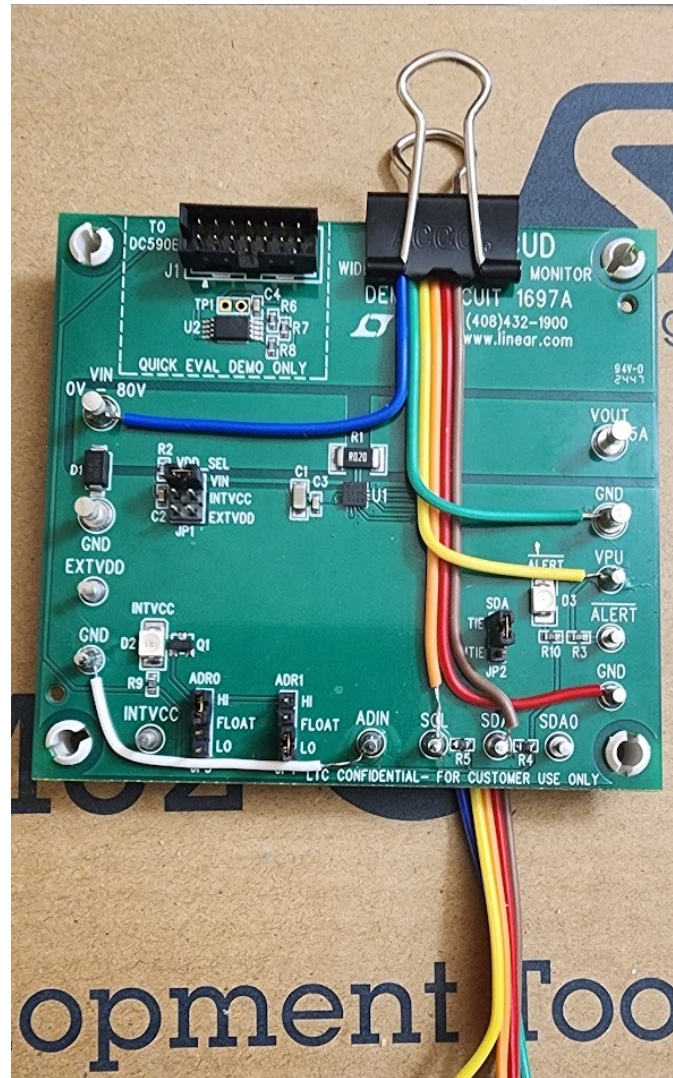


Figure 7.1: DEMO CIRCUIT 1697A Eval Board with Original Shunt, fly leads to connect to Arduino.



Figure 7.2: DEMO CIRCUIT 1697A Eval Board with 17Ω Shunt, fly leads to connect to Arduino, and $4.7k\Omega$ test load.

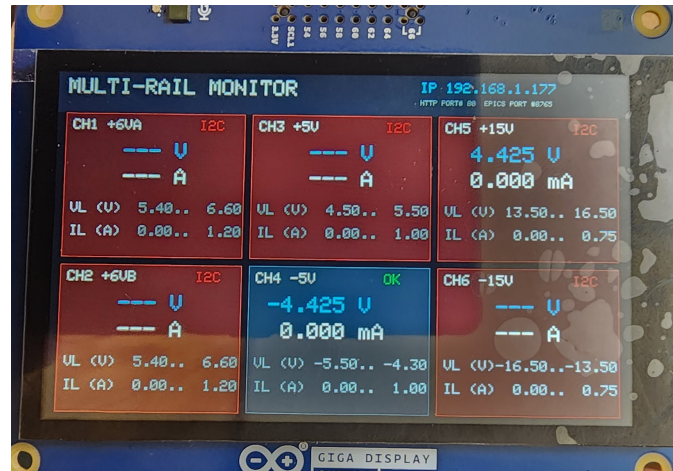


Figure 7.3: Arduino Giga with GIGA DISPLAY SHIELD showing dummy -5V rail being monitored, without alarm

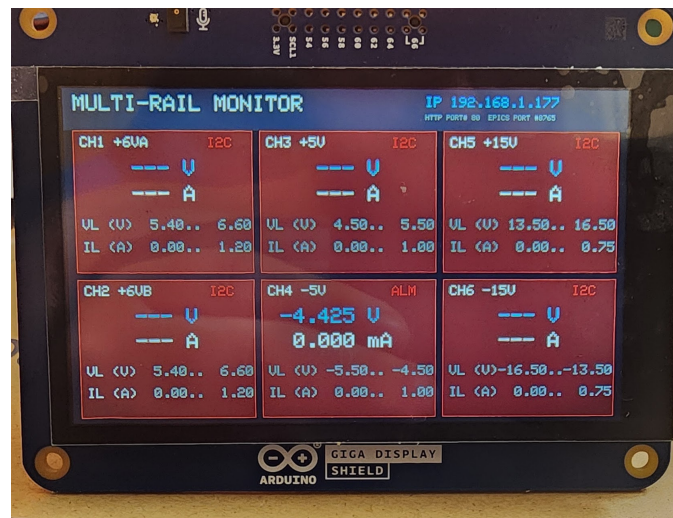


Figure 7.4: Arduino Giga with GIGA DISPLAY SHIELD showing dummy -5V rail being monitored, with UVL alarm, other channels I2C ALARM.

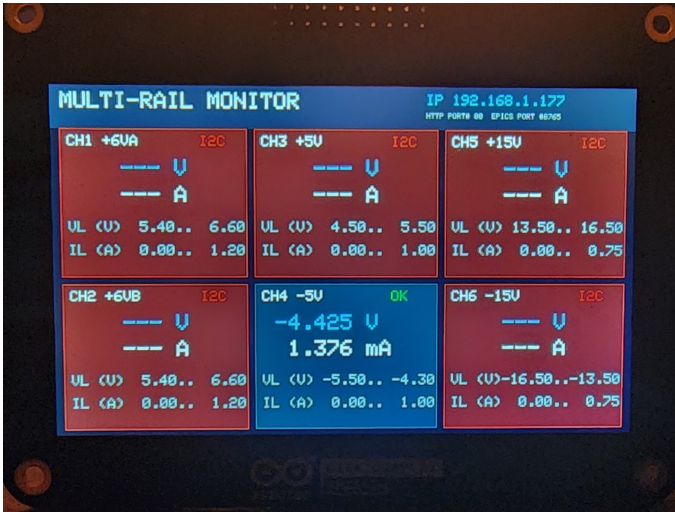


Figure 7.5: Arduino Giga with GIGA DISPLAY SHIELD showing dummy -5V rail being monitored, without alarm, 3.3kΩ Load



Figure 7.6: Arduino Giga with GIGA DISPLAY SHIELD showing dummy -5V rail being monitored, without alarm, 3.3kΩ Load



Figure 7.7: Arduino Giga with GIGA DISPLAY SHIELD connected to an LTC2945 eval board

Appendix A

Design Calculations and Circuit Analysis

This appendix contains the electrical calculations used to select and verify the preliminary hardware. In addition to nominal values, voltage, current, power, logic-level compatibility, and component capability are considered so that the design does not intentionally operate a component beyond its specified limits.

A.1 Current-Sense Shunt Resistor Sizing

The LTC2945 measures the differential voltage between SENSE+ and SENSE– with a full-scale range of 102.4 mV. The ideal shunt resistance for a specified maximum measurable current is therefore

$$R_{SHUNT,ideal} = \frac{V_{SENSE,FS}}{I_{MAX}} = \frac{102.4 \text{ mV}}{I_{MAX}}. \quad (\text{A.1})$$

To prevent saturation, a scaling factor, K_H , is applied to leave some headroom:

$$R_{SHUNT} = K_H \frac{102.4 \text{ mV}}{I_{MAX}}, \quad 0 < K_H < 1. \quad (\text{A.2})$$

For example, $K_H = 0.80$ reserves 20% headroom for overload, tolerance, and transient conditions.

The shunt power at maximum continuous current is

$$P_{SHUNT} = I_{MAX}^2 R_{SHUNT}. \quad (\text{A.3})$$

The installed resistor power rating should exceed this value.

A.1.1 Current Measurement Resolution

The LTC2945 current-sense ADC has a nominal sense-voltage resolution of 25 μV per LSB. The current represented by one ADC count is

$$I_{LSB} = \frac{25 \mu\text{V}}{R_{SHUNT}}. \quad (\text{A.4})$$

This relationship provides an important design tradeoff: a smaller shunt lowers power dissipation but also increases the minimum current increment represented by one ADC count.

A.1.2 Production Shunt Design Table

Table A.1: Production current-sense resistor calculations.

Rail	I_{MAX} (A)	R_{ideal} (Ω)	Selected R (Ω)	V_{SHUNT} (mV)	I_{LSB} (mA/LSB)	P_{MAX} (W)
+5VA	2.00	0.0512	0.039	78.0	0.641	0.156
+5VB	2.00	0.0512	0.039	78.0	0.641	0.156
+5V	1.00	0.1024	0.050	50.0	0.500	0.050
-5V	1.00	0.1024	0.050	50.0	0.500	0.050
+15V	1.00	0.1024	0.050	50.0	0.500	0.050
-15V	1.00	0.1024	0.050	50.0	0.500	0.050

A.1.3 Evaluation-Board Shunt Calculation

The prototype LTC2945 evaluation board was tested from a 5 V source with a load range of approximately 1 k Ω to 10 k Ω . This is a low-current bench-test condition and is separate from the production shunt design.

At the minimum load resistance, the shunt required to place the current-sense input at full scale is

$$R_{S,FS} = \frac{V_{SHUNT}R_L}{V_S - V_{SHUNT}} = \frac{(0.1024 \text{ V})(1 \text{ k}\Omega)}{5 \text{ V} - 0.1024 \text{ V}} = 20.91 \Omega. \quad (\text{A.5})$$

Applying approximately 15% headroom gives

$$R_S = (0.85)(20.91 \Omega) \approx 17.8 \Omega. \quad (\text{A.6})$$

At a 10 k Ω load,

$$I_{MIN} = \frac{5 \text{ V}}{10 \text{ k}\Omega + 17.8 \Omega} \approx 499.1 \mu\text{A}, \quad (\text{A.7})$$

and therefore

$$V_{SHUNT,MIN} = I_{MIN}R_S \approx (499.1 \mu\text{A})(17.8 \Omega) \approx 8.88 \text{ mV}. \quad (\text{A.8})$$

The prototype sense voltage therefore remains inside the LTC2945 102.4 mV full-scale range over the intended test loads.

A.2 PCB Trace and Copper-Pour Current Capacity

Power-path copper must carry the maximum rail current without excessive temperature rise or voltage drop. KiCad's track-width calculator uses a closed-form fit to IPC-2152 data. For an external trace it can be expressed as

$$\Delta T = KI^a W^b T_h^c, \quad (\text{A.9})$$

where I is current in amperes, W is trace width in mils, T_h is copper thickness in mils, and for external tracks the KiCad coefficients are

$$K = 215.3, \quad a = 2, \quad b = -1.15, \quad c = -1.0. \quad (\text{A.10})$$

Solving for minimum width,

$$W_{MIN} = \left(\frac{\Delta T}{KI^a T_h^c} \right)^{1/b}. \quad (\text{A.11})$$

For 1 oz copper ($T_h \approx 1.378$ mil) and a preliminary 10°C allowed rise, the calculated external-trace widths are approximately 3.27 mil at 0.5 A, 10.9 mil at 1 A, and 36.4 mil at 2 A. These are thermal minimums only; the actual layout should use wider traces or copper pours where space permits.

A.3 I²C Pull-Up Resistor Sizing

The I²C bus buffer and I²C line driver both have internal active current sources that eliminate the need for external pullups. Footprints were placed for 4.7k pullups but are set as Do Not Populate.

A.4 LTC2945 Voltage Measurement Resolution

The LTC2945 rail-voltage channel has a nominal full-scale range of 102.4 V and a nominal resolution of 25 mV per LSB. The quantization-only percentage of a measured rail voltage is therefore

$$\epsilon_Q = \frac{25 \text{ mV}}{V_{RAIL}} \times 100\%. \quad (\text{A.12})$$

For example, one count corresponds to approximately 0.50% of a 5 V rail and 0.17% of a 15 V rail. This is only ADC resolution and not total measurement accuracy.

A.5 Negative-Rail Sign Convention

The -5 V and -15 V supplies are electrically isolated from the controller. The monitor can therefore be placed in the high side of each isolated rail and read a positive local differential

quantity even when the system-level rail is labeled negative. Firmware applies the configured sign after acquisition:

$$V_{DISPLAY} = S_V V_{MEASURED}, \quad S_V \in \{+1, -1\}. \quad (\text{A.13})$$

The same channel metadata is used by the local display and network interface so that a negative rail is represented consistently throughout the system.

A.6 Alarm-Limit Scaling

User alarm thresholds are stored and compared in engineering units. For a lower voltage limit V_L and upper voltage limit V_H , the normal voltage state is

$$V_L \leq V_{MEAS} \leq V_H. \quad (\text{A.14})$$

Similarly, for lower and upper current limits,

$$I_L \leq I_{MEAS} \leq I_H. \quad (\text{A.15})$$

If raw ADC thresholds are ever written directly to the LTC2945, the corresponding register value must be calculated from the ADC LSB size. For the current-sense channel,

$$N_I = \frac{I_{LIMIT} R_{SHUNT}}{25 \mu\text{V}/\text{LSB}}, \quad (\text{A.16})$$

and for the rail-voltage channel,

$$N_V = \frac{V_{LIMIT}}{25 \text{mV}/\text{LSB}}. \quad (\text{A.17})$$

A.7 Controller-Supply Power Budget

A preliminary controller power budget was developed using datasheet maximum values where available and conservative design-current allowances for assemblies where a complete board-level maximum current is not specified. A 25% design margin is applied to the calculated load.

Table A.2: Preliminary controller-side power budget.

Load	Supply (V)	Design Current (A)	Power (W)
Arduino GIGA R1 WiFi	5.0	0.500	2.500
GIGA Display Shield	3.3	0.250	0.825
W6100 Ethernet interface	3.3	0.265	0.875
I ² C isolation/controller circuitry	3.3	0.065	0.215
Subtotal	—	—	4.415
25% design margin	—	—	1.104
Total	—	—	5.519

A.8 Component Capability Checks

The calculations above establish the required electrical values. The following checks compare those values with the capability of the selected components.

A.9 Current-Sense Resistor Power Margin

The selected LVK-series 39 m Ω and 50 m Ω four-terminal shunts are rated at 0.75 W. At the maximum design current of 2 A, the 39 m Ω resistor dissipates

$$P = (2 \text{ A})^2(0.039 \Omega) = 0.156 \text{ W}. \quad (\text{A.18})$$

This is approximately 20.8% of the 0.75 W rating, corresponding to about a 4.8:1 power-rating margin before considering PCB temperature and derating. At 1 A, a 50 m Ω shunt dissipates only 0.050 W, approximately 6.7% of a 0.75 W rating.

A.10 LTC2945 Input and ADC Range

The highest monitored nominal rail is 15 V, far below the LTC2945 0–80 V rail monitoring range. The selected production shunts produce 78 mV at 2 A on the 39 m Ω channels and 50 mV at 1 A on the 50 m Ω channels. Both values are below the 102.4 mV current-sense full scale, leaving approximately 24.4 mV and 52.4 mV of ADC headroom respectively. The LTC2945 can operate from an external 2.7–5.9 V supply, so a 3.3 V sensor-side logic supply is within its specified operating range.

A.11 Microcontroller I/O Compatibility

The Arduino GIGA R1 WiFi operates with 3.3 V logic and specifies 8 mA maximum DC current per I/O pin. The controller-side I²C interface is a logic interface, not a load-power interface. The TCA9803 used in the design supports a 0.8–3.6 V A-side supply and a 1.65–3.6 V B-side supply and provides an internal B-side current source of up to 3 mA. A 3 mA bus-low current is below the 8 mA GIGA per-pin limit, providing more than a factor of two current margin. The GIGA pins therefore do not directly source the optocoupler LED or rail-load current; the external interface circuitry performs the required buffering and isolation.

A.12 Digital-Isolation Margin

The ACPL-064L digital optocoupler is specified for 3.3 V/5 V operation and is UL 1577 recognized for 3750 V_{RMS} isolation for one minute. The largest normal potential difference

between the nominal monitored rails is only 30 V from -15 V to +15 V. The optocoupler rating therefore greatly exceeds the functional isolation voltage required by this low-voltage monitoring system. This comparison verifies component capability; it is not intended to redefine the safety rating of the complete assembled product, which also depends on PCB creepage, clearance, connectors, wiring, and enclosure construction.

A.13 3.3 V Regulator Loading

The TPS62912 is rated for up to 2 A output. Using the Week 1 design-current allowances for the display, Ethernet interface, and controller-side isolation circuitry gives an estimated 3.3 V load of

$$I_{3.3} = 0.250 + 0.265 + 0.065 = 0.580 \text{ A.} \quad (\text{A.19})$$

This is approximately 29% of the regulator's 2 A output rating and leaves substantial current margin for transient and implementation variation.

A.14 Tolerance and Measurement-Uncertainty Considerations

The production current-sense resistors have a 1% resistance tolerance. Because current is calculated from $I = V_{SHUNT}/R_{SHUNT}$, an uncorrected 1% shunt error produces approximately a 1% gain error in calculated current. On a 2 A range, this corresponds to approximately 20 mA, which is much larger than the 0.641 mA quantization step of the 39 mΩ channel. The LTC2945 also has its own conversion error; therefore component tolerance rather than ADC quantization is an important contributor to absolute current accuracy. If higher accuracy is required, the actual shunt resistance can be measured and a per-channel calibration factor stored in firmware.

A conservative worst-case linear addition of a 1% shunt tolerance and the LTC2945's less-than-±0.75% total unadjusted conversion error gives an upper-bound estimate of approximately ±1.75% before wiring, thermal, and calibration effects. This estimate is used for design margin rather than as a claim of final calibrated system accuracy.

A.15 Reference Specifications Used for Capability Checks

- Analog Devices, LTC2945 data sheet and product information: <https://www.analog.com/en/products/ltc2945.html>
- Arduino, GIGA R1 WiFi technical specifications: <https://store.arduino.cc/products/giga-r1-wifi>

- Broadcom, ACPL-064L low-power digital optocoupler data sheet/product page: <https://www.broadcom.com/products/optocouplers/industrial-plastic/digital-optocouplers/10mbd/acpl-064l-000e>
- Texas Instruments, TCA9803 I²C buffer data sheet: <https://www.ti.com/product/TCA9803>
- Texas Instruments, TPS62912 buck converter data sheet: <https://www.ti.com/product/TPS62912>
- Ohmite LVK-series current-sense resistor specifications: <https://www.ohmite.com/assets/files/Datasheets/lvk-series.pdf>

Appendix B

Detailed Software Flowchart

This appendix provides the detailed software flow requested for the Week 2 preliminary-design package. It shows the firmware acquisition/alarm loop and the PSC-to-EPICS data path used by the implemented software.

The software for the Multi-Rail Isolated Power Supply Monitoring System is maintained with the hardware design files in the project Git repository:

https://github.com/capotosto/PS_Monitor

The repository provides version control for the Arduino GIGA firmware, hardware-design files, and EPICS IOC software used by the monitoring system. The embedded firmware is maintained under the `firmware` portion of the repository, while the EPICS IOC and associated database/configuration files are maintained under `epics-ioc`. The flowcharts in this section describe the major software operations implemented by these portions of the project.

The Arduino GIGA firmware performs system initialization, acquires voltage and current measurements from the six LTC2945 monitor channels, evaluates alarm conditions, updates the local display, and publishes data through the Ethernet interface. The primary firmware execution sequence is:

1. Initialize the GIGA Display, persistent settings, Ethernet interface, I²C interface, and channel configuration.
2. Load stored network parameters and voltage/current alarm limits from persistent storage.
3. Initialize communication with each of the six LTC2945 monitoring channels.
4. Enter the continuous acquisition and communication loop.
5. Poll each LTC2945 channel for voltage and current measurements.

6. Verify I²C communication status for each channel.
7. If communication fails, mark the associated channel with an I²C error and prevent invalid measurement data from being used.
8. Convert valid raw measurement data to engineering units and apply the appropriate channel polarity and calibration/scaling information.
9. Compare each voltage and current measurement with its configured upper and lower alarm limits.
10. Update the GIGA Display when measurement values, alarm states, or communication states change.
11. Publish the current measurements and status information through the Ethernet PSC protocol interface.
12. Service incoming configuration requests and store modified settings when required.
13. Repeat the acquisition and communication sequence continuously.

B.1 Arduino GIGA Firmware Flowchart

Figure B.1 shows the top-level execution flow of the Arduino GIGA firmware. Following initialization, the controller continuously acquires data from each monitoring channel, verifies communication, converts valid measurements to engineering units, evaluates alarm limits, updates the local display, and services the Ethernet/PSC interface.

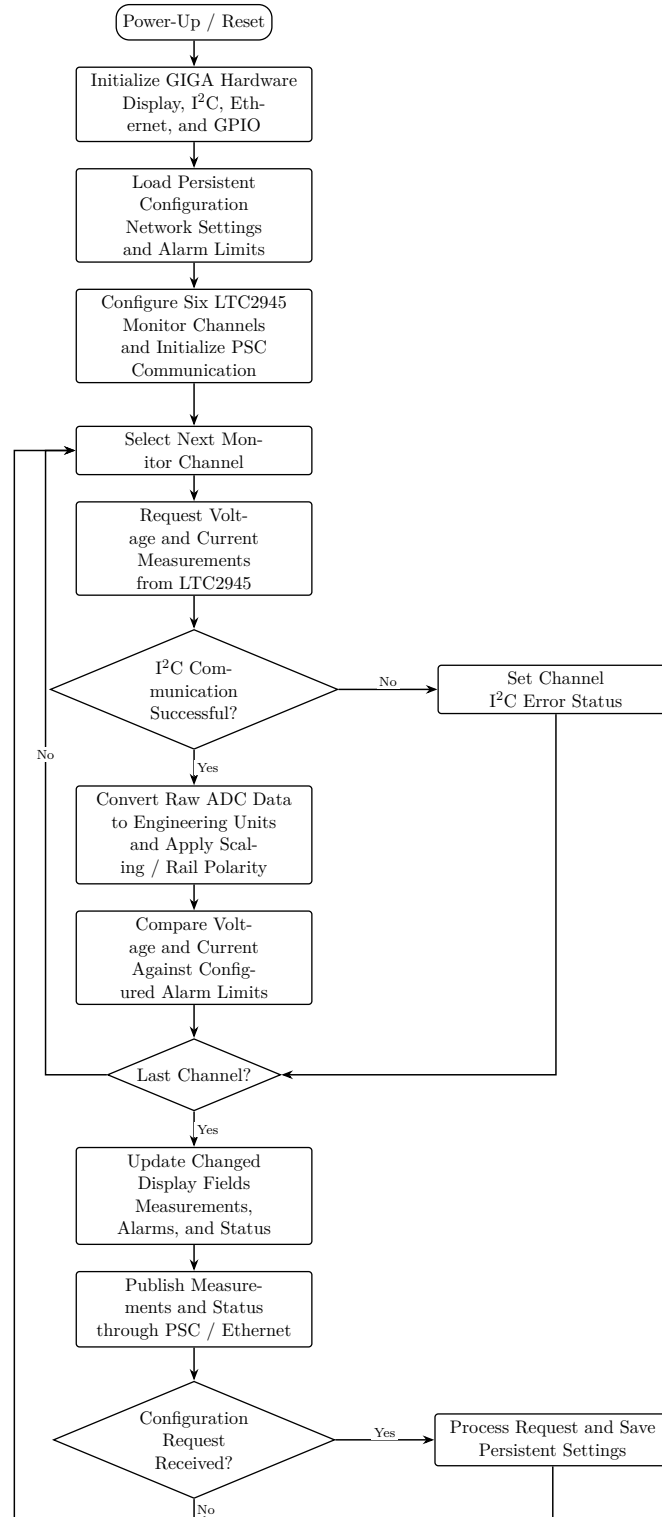


Figure B.1: Top-level Arduino GIGA firmware flowchart. Following initialization, the controller continuously acquires measurements from the six LTC2945 monitoring channels, validates I²C communication, converts valid data to engineering units, evaluates configured alarm limits, updates the local display, publishes measurements through the PSC/Ethernet interface, and services configuration requests.

B.2 PSC and EPICS Communication Flow

The Arduino GIGA does not directly implement EPICS Channel Access. Instead, measurement and status information is transmitted through the Ethernet interface using the PSC protocol. The EPICS IOC uses PSCDriver to communicate with the GIGA, maps the received values into EPICS process variables, and makes those process variables available to the CS-Studio Phoebus operator interface.

This separation allows the embedded controller to remain responsible for hardware acquisition and local alarm evaluation while the IOC provides the interface between the embedded hardware and the facility control system. Figure B.2 summarizes this software data path.

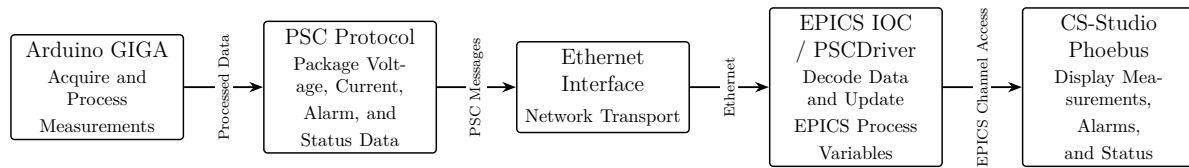


Figure B.2: Software communication path from the Arduino GIGA firmware through PSC-Driver and the EPICS IOC to the CS-Studio Phoebus operator interface.

B.3 Software Configuration Management

Firmware, IOC configuration files, database definitions, and operator-interface files are maintained under Git version control so that hardware and software revisions can be associated with a defined project baseline. The current software implementation and revision history are available in the `PS_Monitor` repository at:

https://github.com/capotosto/PS_Monitor

Changes made during implementation and testing are committed to the repository, providing traceability between the preliminary design presented in this report and the final implemented system. The repository therefore serves as the controlled source for the firmware and control-system software associated with the project.

Appendix C

Updated Component List

The Week 2 component list is intentionally presented in the three-column format requested by the assignment. Quantities represent the present design quantity for one complete monitoring-system hardware set and exclude optional/DNP parts and purchasing spares. Detailed manufacturer and distributor information remains in the KiCad bills of materials.

Table C.1: Updated preliminary component list.

Item	Part Description	Qty.
1	Custom isolated multi-rail sensor PCB	1
2	Custom Arduino GIGA interface/shield PCB	1
3	Arduino GIGA R1 WiFi controller, ABX00063	1
4	Arduino GIGA Display Shield, ASX00039	1
5	LTC2945IUD-1 voltage/current monitor	6
6	ACPL-064L dual-channel digital optocoupler	12
7	TCA9803 I ² C bus buffer/repeater	2
8	P82B715 I ² C bus extender	2
9	TPS62912 low-noise 2 A buck regulator	1
10	W6100-Q Ethernet controller	1
11	SI-46014-F RJ45 Ethernet connector/magnetics assembly	1
12	NTA7002NT1G N-channel MOSFET	6
13	39 m Ω , 1% four-terminal current-sense resistor	2
14	50 m Ω , 1% four-terminal current-sense resistor	4
15	2 k Ω resistor	18
16	4.7 k Ω resistor, populated positions only	8
17	14.7 k Ω resistor	1
18	12.3 k Ω resistor	1
19	10 k Ω resistor	5
20	49.9 Ω resistor	4
21	3.3 Ω resistor	4
22	470 Ω resistor	2

Item	Part Description	Qty.
23	1 M Ω resistor	1
24	0 Ω jumper resistor / populated zero-ohm link	4
25	100 nF ceramic capacitor	37
26	1 μ F ceramic capacitor	12
27	10 μ F ceramic capacitor	2
28	22 μ F ceramic capacitor	5
29	470 nF ceramic capacitor	1
30	22 nF ceramic capacitor	1
31	3.3 μ F capacitor	1
32	1 nF capacitor	1
33	18 pF capacitor	2
34	2.2 μ H power inductor	1
35	120 Ω ferrite bead	3
36	Sensor-board EMI/filter bead (BLE18PS080SN1D)	1
37	25 MHz crystal, FL2500169	1
38	Molex 0039299049 connector/header positions	5
39	Molex 0039289088 connector/header	2
40	Molex 0039289108 connector/header	2
41	Arduino shield connector set	1
42	Keystone 5000 test point	3
43	Keystone 5001 test point	2
44	Keystone 5015 test point	9

This list is expected to change slightly after first-article assembly and final verification. In particular, optional pull-up resistors and other DNP footprints may be populated temporarily for troubleshooting, and component values may be adjusted if testing shows that additional margin or filtering is required.

C.1 Bill of Materials

The current bills of materials for the Sensor Board and Arduino Shield Board are maintained with their respective KiCad hardware-design files in the project `hardware` directory. They are included below for reference.

C.1.1 Sensor Board Bill of Materials

BOM: Power Sensor Board				Production		1 Tape and Reel/Reeling Fees All: No		Add 10% spares for components excluding modules			
Reference	Value	DNP	MPN	Manufacturer	DPN	Distributor	Qty	Qty+10%	Order Qty	Extended Price	
C1,C3,C21,C23,C26,C28,C31,U	CL10A106K88NNNC			Samsung Electro-Mechanics	1276-1860-6-ND	Digikey	12	14	100	\$3.77	
C2,C4,C5,C11,C12,C14,C15,(100n	GRM155R61H104KE14D			Murata Electronics	490-10697-6-ND	Digikey	23	26	100	\$1.41	
C6	22n			Murata Electronics	490-1305-6-ND	Digikey	1	2	100	\$1.01	
C7,C8	10u			TDK	445-181600-6-ND	Digikey	2	3	10	\$3.58	
C9	470n			TDK	445-8020-6-ND	Digikey	1	2	100	\$3.17	
C10,C13,C16,C19,C20	22u			Cal-Chip Electronics, Inc.	4713-GMC21X7S226M10NTDKR-ND	Digikey	5	6	15	\$11.30	
C18	-	DNP				Digikey	1	2			
F81	10n			Murata Electronics	490-BLE18PS080SN1DDKR-ND	Digikey	1	2	10	\$2.30	
H1,H2,H3,H4	MountingHole_Pad					Digikey	4	5			
J101	+6V/A			Molex	WM14518-ND	Digikey	1	2	1	\$2.70	
J102	+6VB			Molex	WM14518-ND	Digikey	1	2	1	\$2.70	
J103	00392890088			Molex	WM3835-ND	Digikey	1	2	1	\$4.58	
J104	+/-15V			Molex	WM14518-ND	Digikey	1	2	1	\$2.70	
J105	+/-5V			Molex	WM14518-ND	Digikey	1	2	1	\$2.70	
J106,J107	39289108			Molex	23-0039289108-ND	Digikey	2	3	1	\$5.02	
J108	Conn_01x04			Molex	WM14518-ND	Digikey	1	2	1	\$2.70	
L1	2u2			74438356022_Wurth Elektronik	732-8232-6-ND	Digikey	1	2	3	\$5.82	
Q1,Q2,Q3,Q4,Q5,Q6	BS170F			NTA7002NT1G	NTA7002NT1GOSDKR-ND	Digikey	6	7	25	\$4.43	
R2,R3,R4,R5,R14,R15,R16,R17,OR		DNP		5108_Keystone Electronics	36-5108DKR-ND	Digikey	8	9			
R6,R12,R13,R33,R34,R35,R36,2k	ERJ-2RK2F2001X			Panasonic Industry	P2.00KLDKR-ND	Digikey	18	20	100	\$1.64	
R7,R18	39mR			Ohmite	273-LVK20R039FERDKR-ND	Ohmite	2	3	5	\$9.45	
R8,R11,R19,R20	50mR			Ohmite	273-LVK20R050FERDKR-ND	Ohmite	4	5	5	\$9.45	
R9,R10	0R			5108_Keystone Electronics	36-5108DKR-ND	Digikey	2	3	10	\$2.01	
R21	0R			Vishay Dale	541-0.01DKR-ND	Digikey	1	2	100	\$0.99	
R22	-	DNP				Digikey	1	2			
R23,R24	4k7			Panasonic Industry	541-4.70KLCI-ND	Digikey	2	3	100	\$1.57	
R25	14k7			Panasonic Industry	P17731CI-ND	Digikey	1	2	10	\$4.52	
R26	4k7			Panasonic Industry	P4.7KPCPT-ND	Digikey	1	2	10	\$4.99	
R27,R28,R29,R30,R31,R32	4k7	DNP		Vishay Dale	541-4.70KLCI-ND	Digikey	6	7			
TP1,TP3	5000			5000_Keystone	36-5000-ND	Digikey	2	3	10	\$2.04	
TP2	5001			5001_Keystone	36-5001-ND	Digikey	1	2	10	\$2.03	
TP4,TP5,TP6,TP7,TP8,TP9	5015			5015_Keystone	36-5015CI-ND	Digikey	6	7	25	\$8.29	
U1,U8,U11,U14,U17,U20	LTC2945IUD-1			Analog Devices Inc.	LTC2945IUD#TRPBFDKR-ND	Digikey	6	7	10	\$110.97	
U2,U3,U9,U10,U12,U13,U15,LACPL-064L	ACPL-064L-500E			Broadcom Limited	516-2614-6-ND	Digikey	12	14	15	\$56.27	
U4	TPS62912			TPS62912RPUR	595-TPS62912RPUR	Mouser	1	2	3	\$7.77	
U5,U6	TCA9803			Texas Instruments	296-47829-6-ND	Digikey	2	3	10	\$6.84	
U7	P82B715DR			Texas Instruments	296-24729-6-ND	Digikey	1	2	5	\$13.55	
TOTAL COST:										\$302.26	

C.1.2 Arduino Shield Board Bill of Materials

BOM: Arduino Shield Board				Production		1 Tape and Reel/Reeling Fees All: No		Add 10% spares for components excluding modules			
Reference	Value	DNP	MPN	Manufacturer	DPN	Distributor	Qty	Qty+10%	Order Qty	Extended Price	
A201	Arduino_Giga_R1_WiFi		ABX00063	Arduino	1050-ABX00063-ND	Digkey	1	2	1	\$69.60	
C6	22n	DNP	GRM155R71H22XA01D	Murata Electronics	490-1305-6-ND	Digkey	1	2	0		
C7,C8	10u	DNP	C2012X7S1E106K1254C	TDK	445-181600-6-ND	Digkey	2	3	0		
C9	470n	DNP	C1005X6R1E474K050B8		445-8020-6-ND	Digkey	1	2	0		
C10,C13,C16,C19,C20	22u	DNP	GMC21X7S226M10NT	Cal-Chip Electronics, Inc.	4713-QMC21X7S226M10NTDKR-ND	Digkey	5	6	0		
C17,C201,C202,C203,C211,C215,C216,C217,C220,C221,C223,C224,C225,C227	100n		GRM155R61H104KE14D	Murata Electronics	490-10697-6-ND	Digkey	14	16	100	\$1.41	
C218	3u3					Digkey	1	2			
C219	1n		1206GC102KAT1A	KYOCERA AVX	478-2956-6-ND	Digkey	1	2	10	\$1.43	
C222,C226	18p		GRM1555CH180JA01D	Murata Electronics	490-5858-6-ND	Digkey	2	3	100	\$1.43	
F81	10n	DNP	BLE18PS080SN1D	Murata Electronics	490-BLE18PS080SN1DDKR-ND	Digkey	1	2			
FB201,FB202,FB203	120R		BBUP00160805121Y00	Pulse Electronics	553-BBUP00160805121Y00CT-NND	Digkey	3	4	25	\$1.97	
J201	SI-46014-F		SI-46014-F	Bel Fuse Inc.	5923-SI-46014-F-ND	Digkey	1	2	1	\$5.81	
J309	0039289088		0039289088	Molex	WM3835-ND	Digkey	1	2	1	\$4.58	
L1	2u2	DNP	74438356022	Wurth Elektronik	732-8232-6-ND	Digkey	1	2	3	\$5.82	
N101	Giga_Display		ASX00039	Arduino	1050-ASX00039-ND	Digkey	1	2	1	\$63.60	
N201	Connector_Set		85	Adafruit Industries	1528-1074-ND	Digkey	1	2	2	\$3.90	
R21	0R	DNP	ERJ-2RKF4701X	Panasonic Industry	P4.70KLDKR-ND	Digkey	1	2			
R23,R24,R205,R206,R207	4K7		CRCW04024K70FKED	Vishay Dale	541-4.70KLCF-ND	Digkey	5	6	100	\$1.57	
R25	15k2	DNP	ERJ-2RKF4701X	Panasonic Industry	P4.70KLDKR-ND	Digkey	1	2			
R26	4k87	DNP	ERJ-2RKF4701X	Panasonic Industry	P4.70KLDKR-ND	Digkey	1	2			
R31,R32	4k7	DNP	ERJ-2RKF4701X	Panasonic Industry	P4.70KLDKR-ND	Digkey	2	3			
R101,R102,R103,R104,R105	10k					Digkey	5	6			
R208	12k3		RN73R1ETTP1232B25	KOA Speer Electronics, Inc.	2019-RN73R1ETTP1232B25DKR-ND	Digkey	1	2	50	\$3.02	
R209,R210,R215,R216	49R9		CRCW040249R9FKED	Vishay Dale	541-49.9LDKR-ND	Digkey	4	5	100	\$1.57	
R211,R212,R213,R214	3R3		CRCW04023R30FKED	Vishay Dale	541-3.30LLDKR-ND	Digkey	4	5	100	\$2.03	
R217	1M		CRCW04021M00FKED	Vishay Dale	541-1.00MLDKR-ND	Digkey	1	2	100	\$1.57	
R218,R219	470R		ERJ-2RKF4700X	Panasonic Industry	P470LDKR-ND	Digkey	2	3	100	\$1.64	
R220	0R		CRCW04020000Z0ED	Vishay Dale	541-0.0DKR-ND	Digkey	1	2	100	\$0.99	
TP1,TP3		2 DNP		5000 Keystone	36-5000-ND	Digkey	2	3			
TP2		1 DNP		5001 Keystone	36-5001-ND	Digkey	1	2			
TP101		1		5000 Keystone	36-5000-ND	Digkey	1	2	10	\$2.04	
TP102		1		5001 Keystone	36-5001-ND	Digkey	1	2	10	\$2.03	
TP201,TP202,TP203		3		5015 Keystone	36-5015CF-ND	Digkey	3	4	10	\$3.70	
U4	TPSG2912	DNP				Digkey	1	2			
U7	P82715DR		P82B715DR	Wiznet	296-24729-6-ND		1	2	2	\$5.42	
U202	W6100-Q		W6100Q	Wiznet	950-W6100Q	Wiznet	1	2	2	\$5.04	
Y201	25MHz		FL2500169	Diodes Incorporated	31-FL2500169DKR-ND	Digkey	1	2	2	\$1.38	
							TOTAL COST:			\$191.55	